國立臺北教育大學理學院數位科技設計學系玩具與遊戲設計碩士班

碩士論文

Department of Digital Technology Design – Master Program in Toy and Game Design

College of Science

National Taipei University of Education

Master's Thesis

設計與實作Android平台上之遊戲控制器

Design and Implementation of Gamepad over Android Platform

蔡華棣

Hua-Di Tsai

指導教授：林仁智博士

Advisor：Jen-Chih Lin, Ph. D.

中華民國 103 年 6 月

June, 2014

謝誌

兩年的碩士生活，一轉眼即將結束。想當初找林仁智老師當指導教授時，還完全不知論文的方向。後來經指導教授的引導以及提供許多論文的方向，這篇論文題目才漸漸地出現。在研究過程中，除了林仁智老師外，俞齊山老師也給了很多寶貴意見，使得論文逐漸步上軌道。

在玩遊所的這兩年，結交了許多不同領域的朋友，在學習的過程中提供了許多不同觀點的想法，讓我也學習到了不同的看法。如今，要離開了這待了六年的系所。而在這的點點滴滴，都是珍貴的回憶。感謝這些時間，陪伴在身邊的朋友們，以及再次感謝我的指導教授與曾經教導過我的老師們。

蔡華棣 謹誌

民國 103 年 7 月

摘要

市面上的 Android 遊戲控制器需要遊戲開發商的配合，而使支援的遊戲數量較多限制。許多智慧型裝置上的遊戲，需要以手指不停的於螢幕上滑動來操作遊戲。對於會流手汗的玩家，手指於螢幕上不停的滑動不但會降低觸控的感應，也容易造成手指的不適。因此本論文製作一遊戲控制器，當遊戲控制器按鈕按下或移動雙軸搖桿時，程式將於 Android 裝置模擬手指觸控螢幕，使遊戲控制器支援更多需要觸控的 Android 平台遊戲，同時也可以支援非遊戲類型的應用程式。此外，玩家使用遊戲控制器於 Android 平台上體驗遊戲時，能減少手指於螢幕觸控時間，降低手指過度頻繁於智慧型裝置螢幕滑動。對於遊戲開發者，也不再受限於遊戲控制器開發商。本論文所開發的遊戲控制器，經過實作驗證，也確實能解決上述問題，對於未來遊戲開發者開發遊戲時也將更為簡單。

關鍵詞：Android、Arduino、RFduino、遊戲控制器、模擬觸控

Abstract

Many smartphone games require the player to slide his or her fingers across the screen. However, smartphone users whose fingers exhibit hyperhydrosis not only reduce the sensitivity capacitive touchscreen but also suffer discomfort when sliding fingers on the screen frequently. Meanwhile, Android gamepads, which are the natural alternative to this problem, do not support too many games, because of a lack of cooperation between gamepad makers and game designers. This study, discusses a gamepad that uses analog sticks and buttons to simulate fingerstrokes, which would thereby facilitate a university compatibility with all Android applications, including games, and bypass problems related to capacitive touchscreen interfaces, including hyperhidrosis.

Keywords：Android、Arduino、Fingerstroke simulation、Gamepad、RFduino

目錄

第一章　緒論 1
　第一節　研究背景與動機 1
　第二節　研究目的 1
　第三節　研究範圍與研究限制 2
　第四節　論文架構 3
第二章　文獻探討 4
　第一節　Android 平台架構 4
　第二節　Android 提供的開發套件 6
　第三節　Arduino 單晶片微電腦 8
　第四節　RFduino 單晶片微電腦 9
　第五節　藍芽通訊 11
　第六節　Android 遊戲控制器 14
　第七節　iOS 遊戲控制器 17
第三章　研究方法 21
　第一節　設計理念 21
　第二節　系統架構 22
　第三節　系統流程 24
　第四節　系統功能 26
第四章　系統實作與結果分析 27
　第一節　系統實作 27
　第二節　結果分析 40
第五章　結論與未來建議 41
　第一節　結論 41
　第二節　未來建議 41
參考文獻 43

表目錄

表 2-1 Bluetooth Link Controller State 13
表 4-1 系統測試項目表 39
表 4-2 Android 智慧型裝置遊戲控制器比較表 40

圖目錄

圖 1-1 論文架構圖 3
圖 2-1 Android 系統架構圖 4
圖 2-2 JNI 作用圖 6
圖 2-3 Android SDK、Android NDK 與 JNI 的關係圖 7
圖 2-4 Arduino UNO R3 正面圖 8
圖 2-5 RFduino 簡介圖 9
圖 2-6 Rfduino DIP 圖 10
圖 2-7 RFduino 開發介面圖 10
圖 2-8 Piconet 示意圖 11
圖 2-9 Scatternet 示意圖 12
圖 2-10 Samsung Android Wireless Gamepad 14
圖 2-11 Samsung Android Wireless Gamepad 按鍵圖 15
圖 2-12 Gametel 16
圖 2-13 標準貼機身控制器 18
圖 2-14 進階貼機身控制器 18
圖 2-15 進階無線控制器 19
圖 2-16 羅技 G550 PowerShell 20
圖 3-1 系統架構圖 22
圖 3-2 系統流程圖 24
圖 4-1 Android 取得事件輸出 28
圖 4-2 Android 單點觸控事件輸出 29
圖 4-3 Android 單點觸控滑動事件輸出 30
圖 4-4 Android 多點觸控事件輸出 30
圖 4-5 Android 觸控座標範圍 31
圖 4-6 模擬觸控流程 32
圖 4-7 Android 遊戲控制器配置應用程式圖 33
圖 4-8 遊戲控制器按鍵配置流程圖 34
圖 4-9 按鈕連接方式示意圖 35
圖 4-10 雙軸搖桿圖 36
圖 4-11 本研究 Android 應用程式藍芽連線流程圖 37
圖 4-12 遊戲控制器測試版本 38

第一章　緒論

第一節 研究背景與動機

隨著智慧型裝置蓬勃發展，原本只用來講電話與傳簡訊的手機，因科技的日新月異，變成人們於日常生活一個不可或缺的隨身物品。而人們也改變了些生活方式，許多人玩遊戲從電腦漸漸轉向智慧型裝置，遊戲開發商也漸漸跟著轉向開發行動平台遊戲。

許多遊戲於行動裝置上遊玩時，操作起來並非那麼的方便。有些遊戲於螢幕上使用類似遊戲控制器的方式操作，而非針對智慧型裝置特別設計不同的操作方式，或是有些遊戲使用智慧型裝置上的感應器操作反而不方便。對於會流手汗的使用者，手指於螢幕上不停的滑動不但會降低觸控的感應，也容易造成手指的不適。

雖然市面上已有支援智慧型裝置的遊戲控制器，但既有的遊戲控制器只支援部分的遊戲。且遊戲控制器若要在遊戲上進行，需要遊戲開發商另外配合遊戲控制器開發商，使得遊戲控制器無法支援太多的遊戲。

因此本研究製作一遊戲控制器，遊戲控制器上的按鍵將模擬手指觸控智慧型裝置的螢幕，並使用藍芽連接至智慧型裝置。如此，遊戲控制器不但能支援更多需要觸控的遊戲，還可以解決頻繁的於螢幕滑動造成手指的不適，以及方便的操作使玩家得到更好的遊戲體驗。

第二節 研究目的

基於上述研究背景與動機，本研究的研究目的如下：

1. 設計遊戲控制器於智慧型裝置的遊戲中使用：

   本研究製作遊戲控制器與智慧型裝置連接，使用者將能以遊戲控制器於智慧型裝置的遊戲中操作。

2. 實作能支援更多 Android 平台遊戲的遊戲控制器：

   本研究使用模擬使用者手指觸控智慧型裝置螢幕的方法，使得本研究遊戲控制器將能支援更多需要觸控的遊戲。

3. 降低手指過度頻繁於智慧型裝置螢幕滑動：

   使用遊戲控制器代替手指於螢幕的觸控，以解決會流手汗的玩家，不停的滑動造成的觸控的感應靈敏度下降以及手指的不適。

4. 將 Android 智慧型裝置擺放至更遠的距離：

   遊戲控制器與 Android 智慧型裝置是以藍芽建立連線，而非以有線的方式連接。當使用者將 Android 智慧型裝置連接於電視或是外接螢幕時，可以將 Android 智慧型裝置擺放至更遠的距離。

第三節 研究範圍與研究限制

就目前智慧型裝置中市場佔有率最高的 Android 平台與 iOS 平台而言，由於 Android 平台為開放原始碼的平台，開發者要於 Android 平台上開發較底層的程式時較不會受到限制，而 iOS 平台只開放開發者開發一般的應用程式，對於開發者所能開發的應用程式限制也相當的多。本研究的模擬觸控部分需要撰寫底層的程式，故本研究的範圍為自由度較高的 Android 平台。

本研究以模擬觸控的方式，雖然使遊戲控制器於 Android 平台上支援更多需要觸控的遊戲，但模擬觸控於 Android 平台上開啟時，程式會向使用者要求更高的權限，而智慧型裝置的原廠設定，無法讓使用者可以更改應用程式的權限。本研究手機與遊戲控制器的藍芽連接的部分，目前 Android 4.3 版本以上才支援藍芽 4.0 的程式。因配備藍芽 4.0 版本以下的型裝置無法與藍芽 4.0 的裝置連接，所以 Android 智慧型裝置本身也須支援藍芽 4.0，才能與本研究遊戲控制器連接。

故本研究的限制為 Android 版本需要為 Android 4.3 以上、支援藍芽 4.0 以及能更改應用程式權限的 Android 裝置。

第四節 論文架構

本論文共分五個章節，第二章文獻探討，介紹使用到的技術及既有的智慧型裝置遊戲控制器。第三章研究方法，介紹本研究的系統流程、本研究的系統架構、系統實作與詳細的說明本研究實作的方法。第四章系統實作與結果分析說明本研究的實驗結果，並進一步的分析本研究與既有的智慧型裝置遊戲控制器之差異。第五章結論與未來建議，說明本研究的研究結果，進一步討論未來可以發展的空間與方向。本論文架構如圖 1-1 所示。

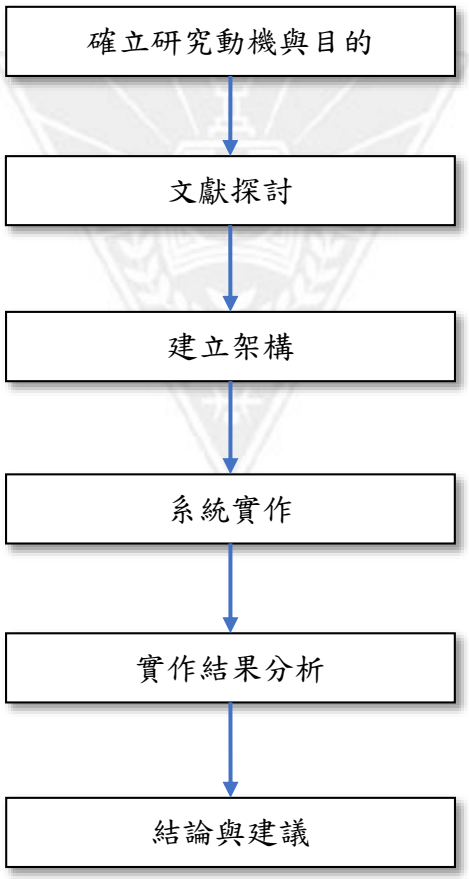


圖 1-1 論文架構圖

第二章　文獻探討

本章第一節介紹 Android 架構，第二節為 Android 提供的開發套件架構及說明，第三節介紹何謂 Arduino，第四節介紹何謂 RFduino 及 RFduino 的規格與功能，第五節簡介藍芽技術，第六節開始介紹與本研究相關的遊戲控制器。

第一節 Android 平台架構

Android 是基於 Java 程式語言與 Linux 作業系統核心的一個智慧型行動裝置平台。Android 最初是由 Andy Rubin 於 2003 年所開發的，後來 Andy Rubin 的 Android 公司被 Google 所收購而延續開發，且於 2007 年 11 月公布。Android 系統使用了開放免費原始碼許可證，大部分是以 Apache License 發行，而核心 Linux 部分則是以 GNU General Public License 許可。

Android 系統架構如圖 2-1 所示，分別為 Java code (Applications、Application Framework)、系統函式庫(External Libraries、Hardware Abstraction Layer)、虛擬機器(Android Runtime) 及核心層(Linux Kernel)。

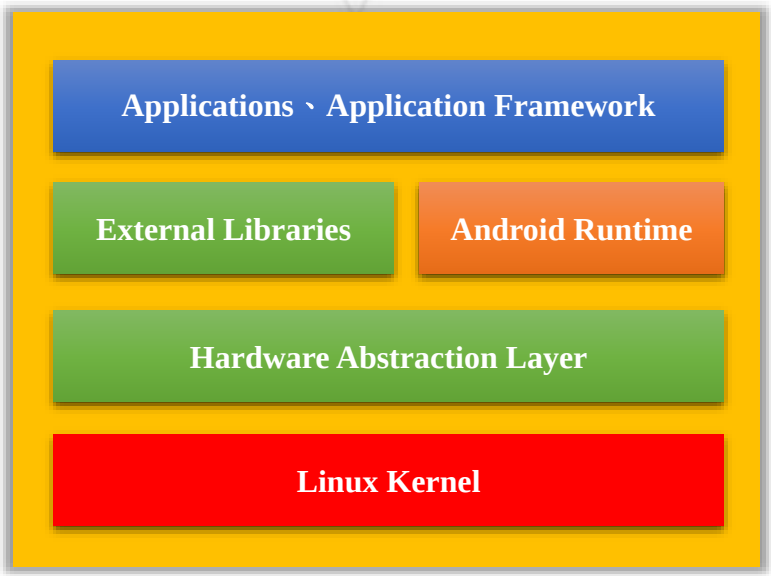


圖 2-1 Android 系統架構圖

Android 系統架構中的 Application 包含了原生的 APK 與第三方的 APK，而每個 APK 都是以 Application Framework 為基礎而進行開發的，且 APK 由 Android Runtime 執行運作。

Android 系統架構中的 Application Framework 提供了數種系統相關的類別，提供使用者開發應用程式、使用者介面管理，也提供與硬體連結的抽象方法。

Android 系統架構中的 Android Runtime 用於驅動與執行 APK，Android Runtime 會連結 APK 與相關的函式庫，Android Runtime 為組成 Application Framework 的基礎。

Android Runtime 包含了 Core Libraries 與 Dalvikav Virtual Machine。Dalvikav Virtual Machine 最佳化與確保每個程式於 Android 裝置上能有效的運作，而執行緒與記憶體的管理則是基於 Linux Kernel。Core Libraries 使開發人員能以 Java 程式語言匯入各種函式庫來撰寫 Android 應用程式。

Android 系統架構中的 Libraries 是基於 Linux Kernel 運作，其中包含了許多以 C/C++程式語言所撰寫的函式。Libraries 中有撥放多媒體的 Media Framework 與 Audio Manager、顯示畫面與管理面的 Surface Manager、負責繪圖的 OpenGL、OpenGS 與 SGL、SQLite 行動資料庫、管理網頁瀏覽與網路安全的 WebKit 及 SSL。

Android 系統架構中的 Hardware Abstract Layer 使開發人員可以在不必撰寫與 Linux Kernel 驅動程式作連結的程式、不考慮硬體驅動程式的情況下進行各類硬體的開發。而 Android 系統架構中的 Linux Kernel 負責驅動硬體、執行緒管理、記憶體管理、安全性、網路以及電源控制等的管理。

第二節 Android 提供的開發套件

Android NDK (Native Development Kit) 為 Android 提供的一個開發套件，Android NDK 允許開發者開發時使用 native-code 的語言，如 C/C++來實作 Android 應用程式的一部分。Android NDK 的使用時機為必須提高性能時、使用由 C/C++ 編譯的第三方類別庫(如 OpenCV)，或是 Android 底層的應用程式設計。

JNI (Java Native Interface) 於 JAVA SDK 1.1 時推出，為一種 programming framework，目的是為了使執行於 Java Virtual Machine (JVM) 的 Java 程式能夠呼叫 Native Library，或是被 Native Application 所呼叫。如圖 2-2 所示，JNI 就像是 Java code 與 Native code 互相溝通的橋樑。

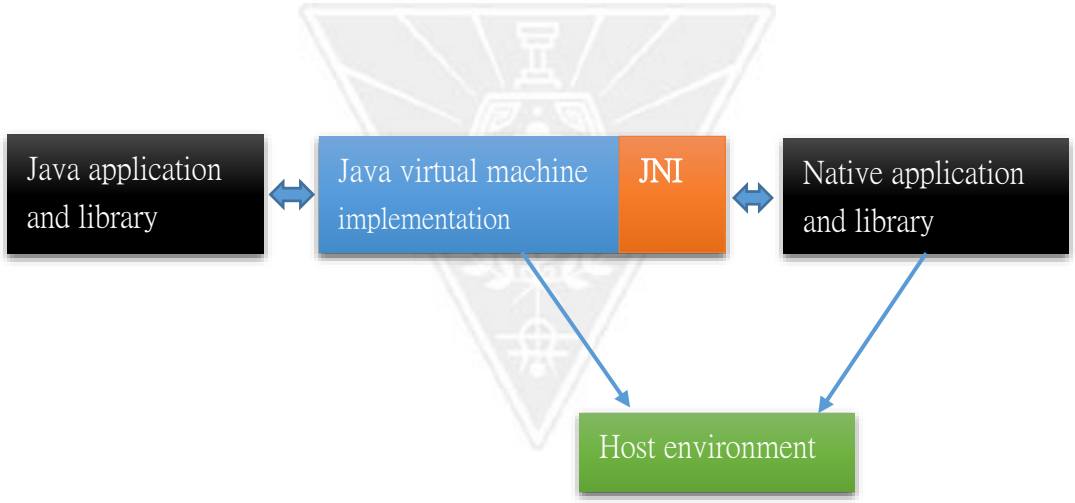


圖 2-2 JNI 作用圖

Android 中，Android SDK、Android NDK 與 JNI 的關係如圖 2-3 所示，於 Android SDK 中的 Java code 以 JNI 呼叫 shared library (以 Android NDK 編譯為 so 檔的 C/C++ code)，最後再以 Android SDK 打包輸出為 APK 檔。

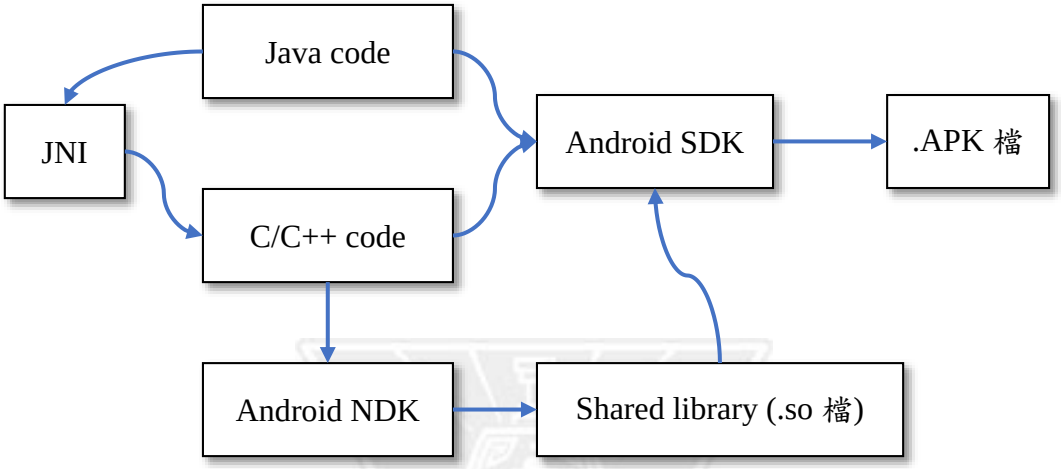


圖 2-3 Android SDK、Android NDK 與 JNI 的關係圖

於 Android 平台上，以 Android SDK 撰寫 Java 程式，並呼叫許多 Android API 來實作各種功能，為了大量增進效能或是呼叫底層函式，才需要使用到 Android NDK。Android NDK 以 JNI 開發，把 C/C++撰寫的程式碼轉換為 Java 所能認得的。最後再以最後再以 Android SDK 打包輸出為 APK 檔，於 Android 裝置中使用。

第三節 Arduino 單晶片微電腦

Arduino 是一個靈活、易於使用的軟體與硬體的開放原始碼電子原型開發平台，可適用於藝術家、設計師、愛好者和有興趣創造互動式物件或互動式環境的人們。圖 2-4 Arduino UNO R3 正面圖。

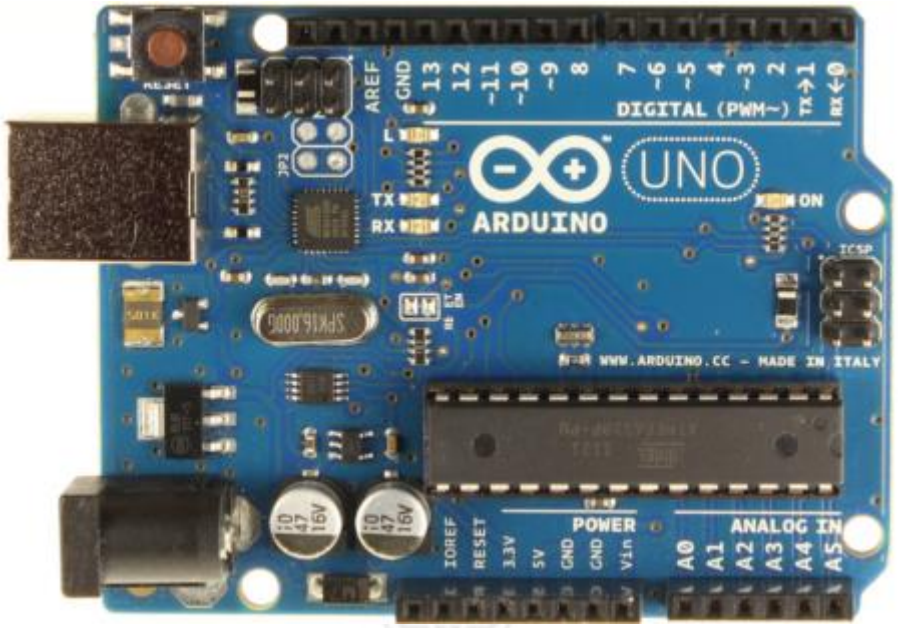


圖 2-4 Arduino UNO R3 正面圖

資料來源：http://arduino.cc/en/Main/arduinoBoardUno

Arduino 不但可以藉著接收各種感測器的輸入訊號來感測各種環境，也可以透過控制燈、馬達或其他裝置來影響周圍的環境。Arduino 電路板上的微控制器使用的是基於 Wiring 的 Arduino 程式語言，而開發環境是基於 Processing 來進行編譯。Arduino 可以單獨的與感測器運行，也可於電腦上使用 Flash、Processing、MaxMSP 等來與 Arduino 進行溝通。

第四節 RFduino 單晶片微電腦

RFduino 為 Arduino 的縮小版，由一家名為 Open Source RF 的電子商店所研發。RFduino 僅指尖大小，配備 Nordic 32 bit ARM Cortex-M0 處理器，七個 GPIO（Digital IO、Analog ADC、SPI、I2C、UART 及 PWM），內建藍芽 4.0 Low Energy 可與任何支援藍芽 4.0 的智慧型裝置溝通。圖 2-5 為 RFduino 簡介圖。

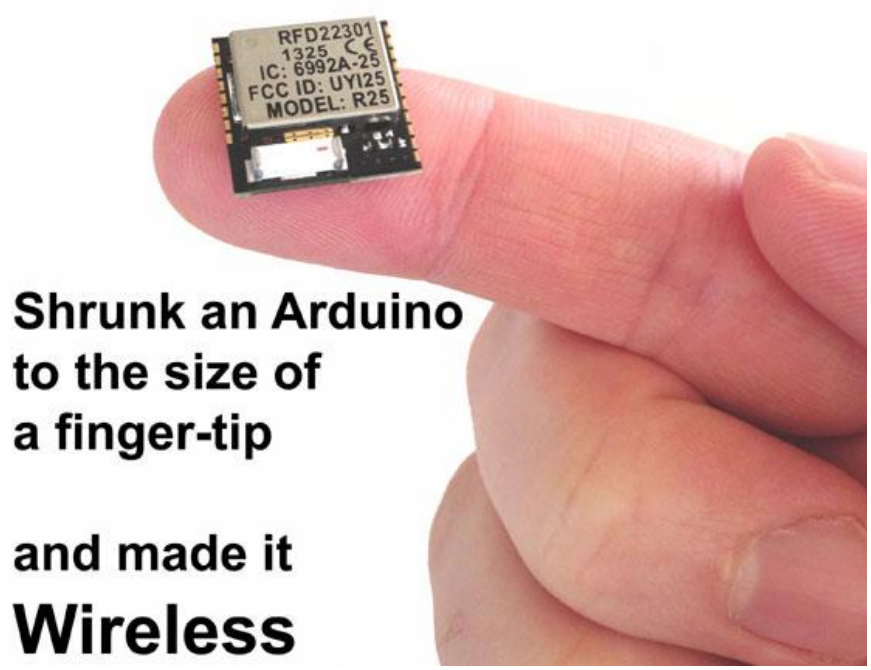


圖 2-5 RFduino 簡介圖

資料來源：http://www.rfduino.com/

RFduino 的開發商還為 RFduino 開發了一些小型的輔助板，使它具備疊加和插入其他設備功能。智慧型裝置通過 RFduino 可以控制一些列設備或物件。它是一款功能更強大尺寸卻極小開發模塊，與 Arduino UNO 或 Arduino DUE 相似，卻比 Arduino UNO 或 Arduino DUE 還要強大、尺寸更小，不需要像 Arduino 另外連接藍芽模塊。如圖 2-6，RFduino 使用的 DIP 能方便的與麵包板 (solderless breadboard)連接。

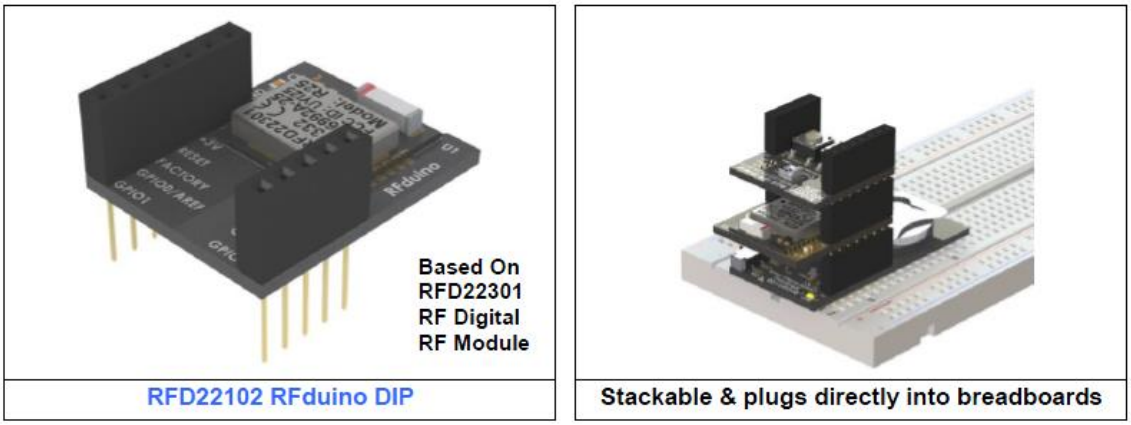


圖 2-6 Rfduino DIP 圖

資料來源：http://www.rfduino.com/wp-content/uploads/2014/03/rfduino.datasheet.pdf

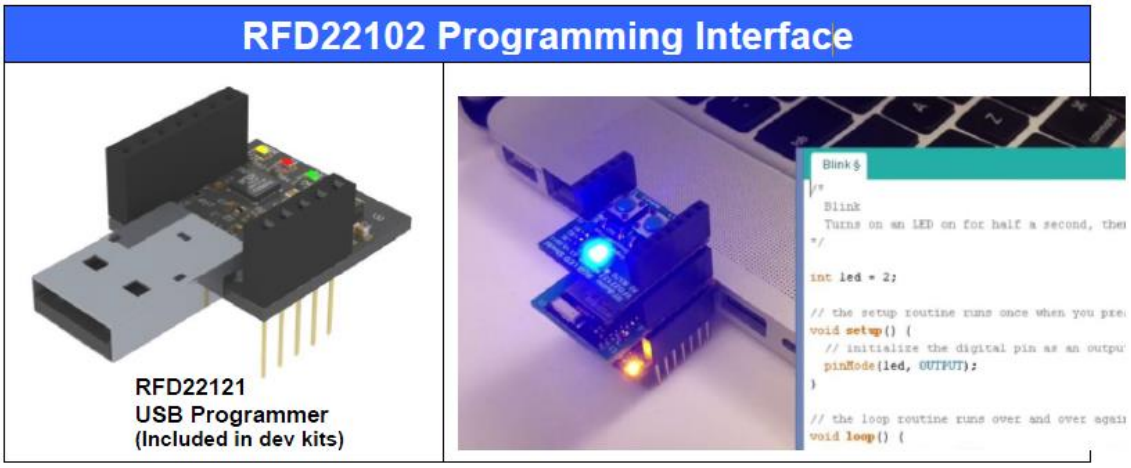


圖 2-7 RFduino 開發介面圖

資料來源：http://www.rfduino.com/wp-content/uploads/2014/03/rfduino.datasheet.pdf

如圖 2-7，RFduino 可藉由 RFduino USB shield 而接上電腦的 USB 連接埠，接著使用 Arduino IDE 來載入 Arduino sketch，RFduino USB shield 將會被自動偵測到，即可安裝自己的專案於 RFduino 上。RFduino 的開發商也為 iPhone 研發了幾個相關的應用程式，而 Android 平台的應用程式將會在未來設計中逐漸增加。

第五節 藍芽通訊

Bluetooth (藍芽)是一種無線傳輸技術，於 2.4GHz 上傳輸，可同時傳送語音與數據。Bluetooth 有著價格便宜、操作容易及功率較低的優點，且能支援多個具藍芽的裝置互相連接。

Bluetooth 可運作於 Master 或 Slave 模式，一個藍芽裝置可以同時為 Master 與 Slave，或在 Master 與 Slave 間切換。發出要求的一端為 Master，而接收端為 Slave。由於 Master 控制整個藍芽網域，當兩個藍芽裝置建立連線後，Master 與 Slave 間切換。每個藍芽裝置都有一個藍芽裝置位址（Bluetooth Address）與時脈。當 Slave 與 Master 建立連結時，Master 會把自己的藍芽裝置位址與時脈通知 Slave，Slave 的基頻會計算出 Master 的跳頻序列，與自己的跳頻序列比較，計算出位移量，以便調整自己的頻率，並與 Master 完成同步的動作。

如圖 2-8，藍芽有 Piconet 與 Scatternet 兩種網路類型，Piconet 的特點為一個 Master 最多可以擁有七個 Slave。

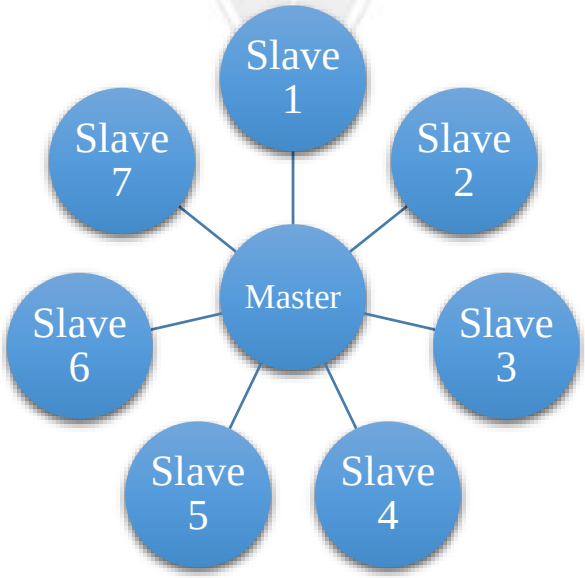


圖 2-8 Piconet 示意圖

如圖 2-9，Scatternet 則是由 Piconet 所組成的，Piconet 的 Slave 能成為另一個 Piconet 的 Master，使得兩個 Piconet 能互相通訊，如此就成為了 Scatternet。

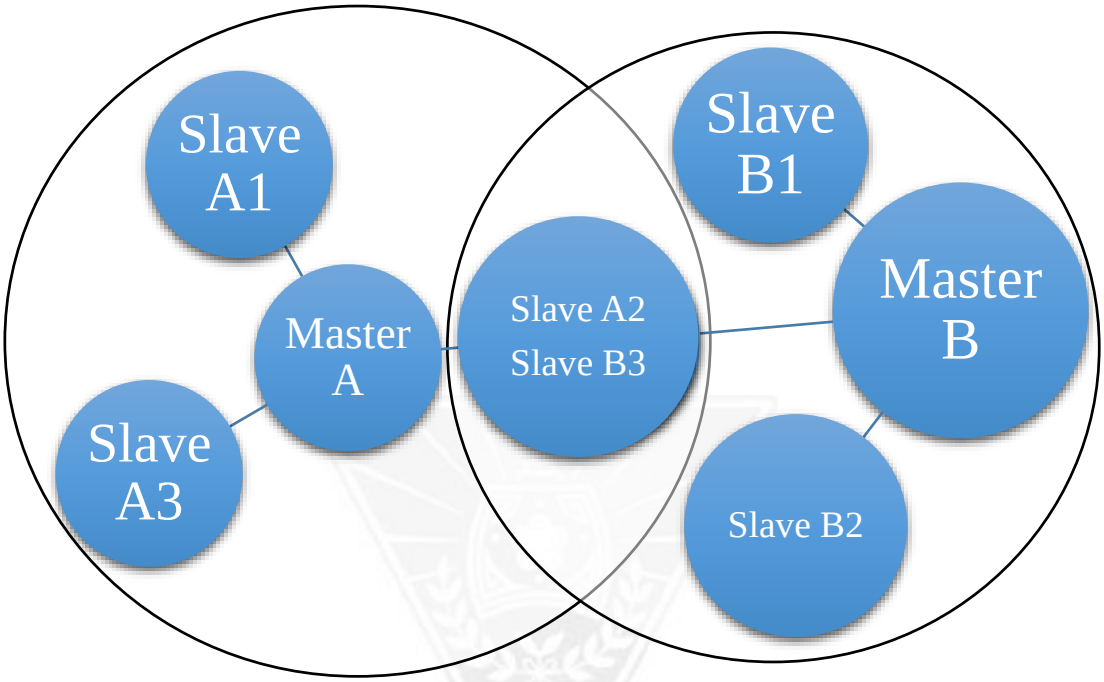


圖 2-9 Scatternet 示意圖

Bluetooth 在不同的連線狀況下會調整連接工作狀態(Link Controller State)以節省裝置的用電量，而 Bluetooth 所調整的狀態共有 Active、Sniff、Hold、Park 四種，當消耗功率低時，會延長 Slave 傳送資料的頻率間隔以節省 Bluetooth 設備的耗電量。

表 2-1 Bluetooth Link Controller State

Active	當 Bluetooth 設備正在互相通訊時的狀態，會持續與其他 Bluetooth 設備 Inquiry 與 Paging。
Sniff	相似於 Active 狀態，Sniff 相較於 Active 的差別為 Sniff 延長了訊號發送的間隔以達到節省耗電量的效果。
Hold	Bluetooth 設備會暫時停止支援 ACL 連線，但仍可進行 SOC 連線。因此不會占用到實體通道的時槽，而提高藍芽網路的運行效率。
Park	不用傳輸資料時會進入此狀態，等待 Master 傳送喚醒的訊息。Master 有週期性的廣播訊號，Parked Member Address 指定某個 Park 狀態的 Slave 時，該 Slave 就會轉到 Active 狀態。

表 2-1 為 Bluetooth 在各種不同的連線狀況下調整連接工作狀態狀態的說明。當 Bluetooth 設備正在互相通訊時會進入 Active 狀態，此狀態下會持續與其他 Bluetooth 設備 Inquiry 與 Paging。Bluetooth 切換為 Sniff 狀態時，Slave 監聽的時間縮短，Master 只要在特定的時槽內傳送資訊給 Slave 即可。於連結的狀態時，Slave 可以進入 Hold 狀態。進入 Hold 狀態前，Master 與 Slave 會協議 Slave 能在 Hold 狀態的時間，當協議的時間到了，Slave 將重新進入 Active 狀態，並與通道連結進行同步、等待 Master 的指令。當 Slave 不需要再參與 Piconet，但仍要與 Piconet 維持同步時，便可以進入 Park 狀態。Park 狀態除了能降低 Bluetooth 設備的消耗功率外，還能使 Master 服務更多的 Bluetooth 設備。

第六節 Android 遊戲控制器

本節將介紹兩個與本研究相似的遊戲控制器，有 Samsung Android Wireless Gamepad 及 Gametel。

- Samsung Android Wireless Gamepad

圖 2-10 為 Samsung 的 Android Wireless Gamepad (EI-GP20)，能將一台 Android 智慧型手機變身為便於操作的掌上型遊戲機。Samsung Android Wireless Gamepad 有一個伸縮的支架，能夠容納 4 吋到 6.3 吋的裝置。而 Samsung Android Wireless Gamepad 連接的部分是使用 Bluetooth 3.0 來與 Android 裝置連接。而 Samsung Android Wireless Gamepad 對於 Android 平台只支援 Android 4.1(Jelly Bean)以上的裝置，目前針對 Android 4.3 以上有特別的優化。



圖 2-10 Samsung Android Wireless Gamepad

資料來源：http://www.samsung.com/

如圖 2-11，Samsung Android Wireless Gamepad 按鍵的部分配備了兩個 Analogue Sticks、4 個 Action buttons、兩個 Trigger buttons、Start 與 Select button。藍芽使用藍芽 3.0 與 HID Bluetooth，並支援 NFC 快速配對以及自動連接的功能。



圖 2-11 Samsung Android Wireless Gamepad 按鍵圖

資料來源：http://www.samsung.com/

當按下 Samsung Android Wireless Gamepad 上的 Play 鍵，Android 裝置就會啟動或下載 Samsung 的 S Console 應用程式。在 Mobile Console 中，使用者能瀏覽支援裝置的遊戲並能進行下載。

Samsung 也提供了對應該遊戲控制器的原始碼。開發者不但能將自己開發的遊戲結合 Samsung 的遊戲控制器，還可以將以結合 Samsung Android Wireless Gamepad 的遊戲上架至 Samsung 的 S Console。

- Gametel

圖 2-12 為 Gametel，是一個藍芽遊戲控制器，支援 Android 及 iOS，也支援 Windows 與 OS X。



圖 2-12 Gametel

資料來源：http://gametel.se/index.php/explore/specification

Gametel 按鍵的部分搭配了四個 Action buttons、四個 Navigation buttons、兩個 Firing buttons、一個 Select button 以及一個 Start button。能夠以藍芽連接，另外也可以使用 USB。平台部分支援 Android 2.1 以上、iOS 4 以上、Windows 7 以上與 Mac OS X Snow Leopard 10.6 以上的版本。支架最大可以原寬度為 72 mm 的裝置。

Gametel 也提供了對應該遊戲控制器的原始碼。Gametel 不但提供遊戲開發者以原生的開發方式開發支援 Gametel 的遊戲控制器，Gametel 還提供了某些遊戲引擎的開發套件。

第七節 iOS 遊戲控制器

2013 年 9 月，Apple 公司開放開發者能設計與開發支援 Apple 產品的遊戲控制器。Apple 公司也為開發支援遊戲控制器的遊戲訂定了規範，使得遊戲設計者與玩家能夠遵循。

Apple 的遊戲控制器規格有三種，分別有標準貼機身控制器、進階貼機身控制器與進階無線控制器。任何遊戲開發者想要開發遊戲控制器都必須符合這三種規格。若要 Apple 的遊戲控制器或支援遊戲控制器的應用程式，必須符合下列規定：

- 使用遊戲控制器為非必要：

  支援遊戲控制器的應用程式時，必須無遊戲控制器的時候也能操作。在 iOS 設備下使用觸控或是感測器，於 OS X 中則是使用滑鼠鍵盤。遊戲控制器只是提升遊戲體驗，不得為必要。

- 連接一次後就會自動連接：

  開發者能於應用程式中自製搜尋與發現遊戲控制器的介面，當遊戲控制器連接或斷開時，系統會出現通知，開發者能藉此通知來切換遊戲的控制方式。

- 按鍵的輸出值必須遵守控制器配置：

  各家遊戲控制器開發商所製作的遊戲控制器不盡相同，但都必須遵守按鍵輸出的配置。應用程式開發者也只需要讀取按鍵輸出的配置，就能夠支援各種支援 iOS 的遊戲控制器。

- 控制器資料快照：

  開發者能在應用程式中先定義幾種遊戲操控方式，使用者也能儲存自定義遊戲方式。

如圖 2-13 所示，標準貼機身控制器(standard form-fitting controller)是將 iOS 裝置放於遊戲控制器中，與控制器合為一體。而標準貼機身控制器含有一組方向鍵(D-pad)、四顆 Action buttons 與兩顆 shoulder button。

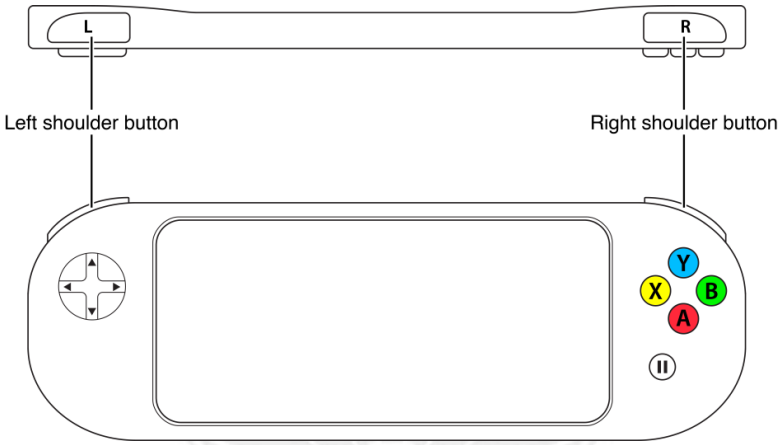


圖 2-13 標準貼機身控制器

資料來源：iOS Developer Library - Game Controller Programming Guide

進階貼機身控制器(extended form-fitting controller)如圖 2-14 所示，比標準貼機身控制器(standard form-fitting controller)多了兩顆 shoulder button 及兩個 Joysticks。

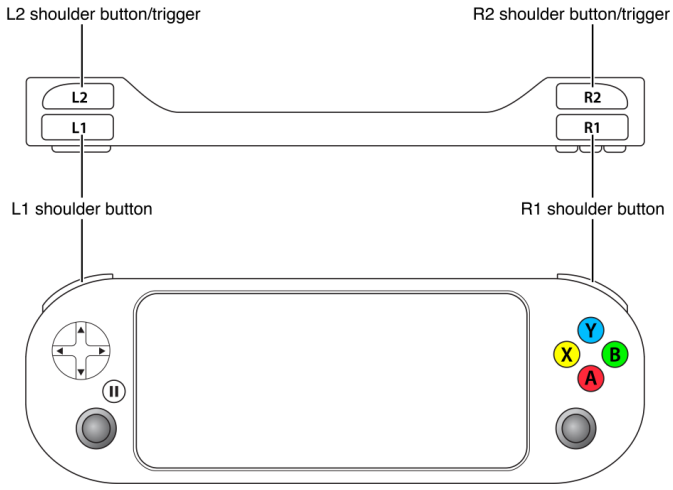


圖 2-14 進階貼機身控制器

資料來源：iOS Developer Library - Game Controller Programming Guide

進階無線控制器(extended wireless controller)如圖 2-15 所示，遊戲控制器與 iOS 裝置是分開的，以無線方式連接並附有 LED 燈。

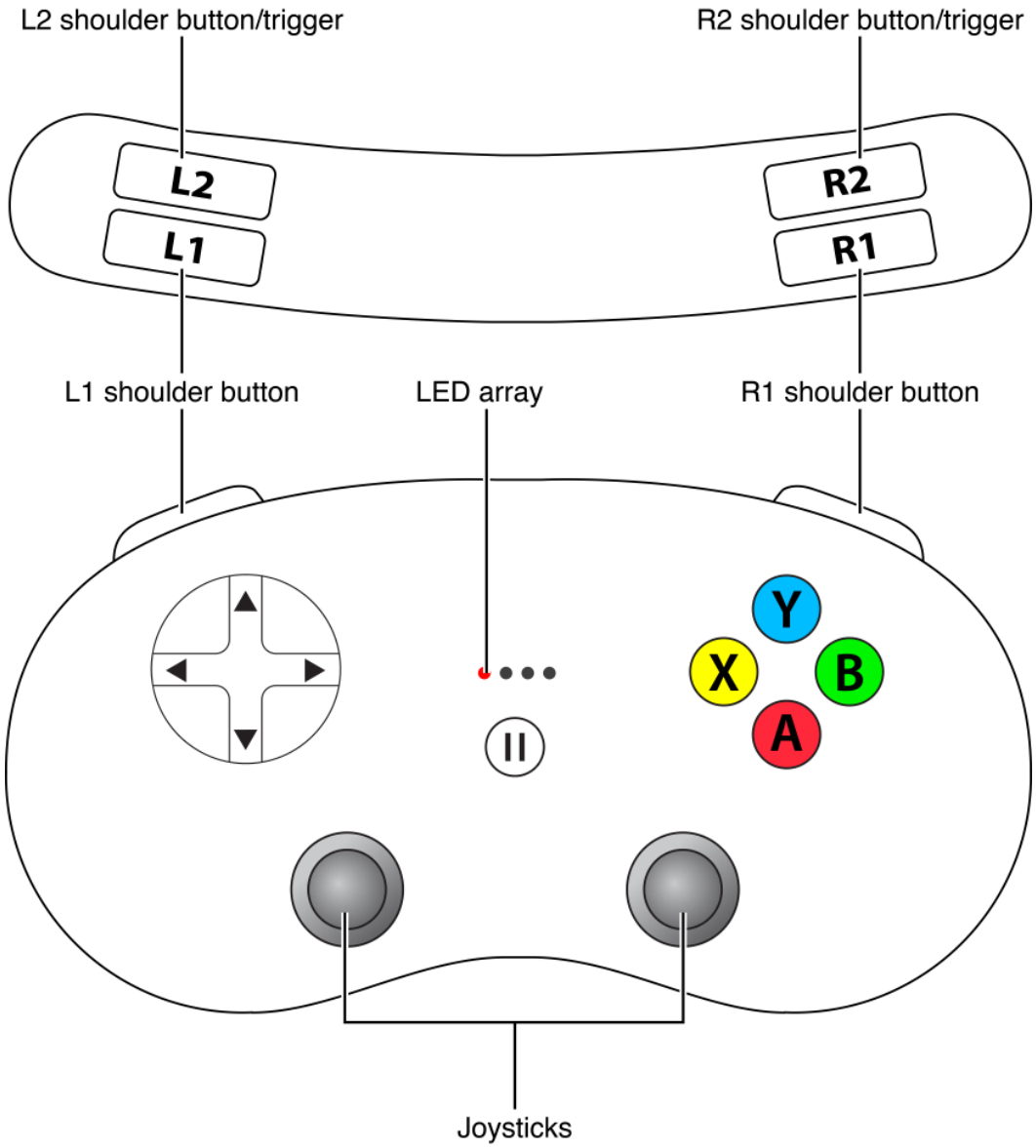


圖 2-15 進階無線控制器

資料來源：iOS Developer Library - Game Controller Programming Guide

2013 年 12 月，羅技(Logitech) 出產了一款名為 G550 PowerShell 的 iOS 遊戲控制器，G550 PowerShell 的外觀如圖 2-16 所示。



圖 2-16 羅技 G550 PowerShell

資料來源：http://gaming.logitech.com/zh-tw/product/powershell-controller-and-battery

PowerShell 按鍵的部分搭配了類比 D-pad、左右發射鍵以及按鍵組，而遊戲控制器內還提供了 1500 mAh 的電池。平台的部分只支援 iOS 7 以上，最低系統需求為 iPhone 5、iPhone 5s 或 iPod touch (第 5 代)。PowerShell 於官方的遊戲控制器規格為標準貼機身控制器(standard form-fitting controller)，標準貼機身控制器是以 iOS 連接頭來連接 iOS 手機，而非使用藍芽連線的方式。

第三章　研究方法

本章共有四節，分別為設計理念、系統架構、系統流程以及本研究系統所提供的功能。

第一節 設計理念

為達到本研究目的－設計遊戲控制器於智慧型裝置的遊戲中使用、實作能支援更多 Android 平台遊戲的遊戲控制器、降低手指過度頻繁於智慧型裝置螢幕滑動、將 Android 智慧型裝置擺放至更遠的距離，本研究的設計理念為：

- 更遠的使用距離：

  當 Android 智慧型裝置為大型，或是連接於螢幕、電視時，Android 裝置便可以成為遊戲主機。此時使用本研究遊戲控制器的藍芽連線方式，就不會因為有線而被限制在較短的距離。

- 簡單明瞭的介面：

  本研究的 Android 應用程式將不會有太過複雜的介面，只要輕鬆點擊幾顆按鈕，就可以做好與遊戲控制器的連線、按鍵配置設定等。

- 能針對不同的遊戲配置按鍵：

  使用者可以針對不同的遊戲，配置並儲存不同的遊戲控制器按鍵設定。

- 能快速切換按鍵配置：

  當更換遊戲時，只要於本研究的 Android 應用程式中讀取遊戲控制器配置，便可以馬上改變按鍵對應於 Android 裝置螢幕的位置。

- 支援更多的遊戲：

  將以模擬使用者手指觸控的方式，以遊戲控制器支援遊戲，而非請遊戲開發商於開發遊戲時支援遊戲控制器。

第二節 系統架構

本系統架構如圖 3-1 所示，分為 Android 應用程式與遊戲控制器兩端，而兩端的溝通是以藍芽做資料的傳遞。

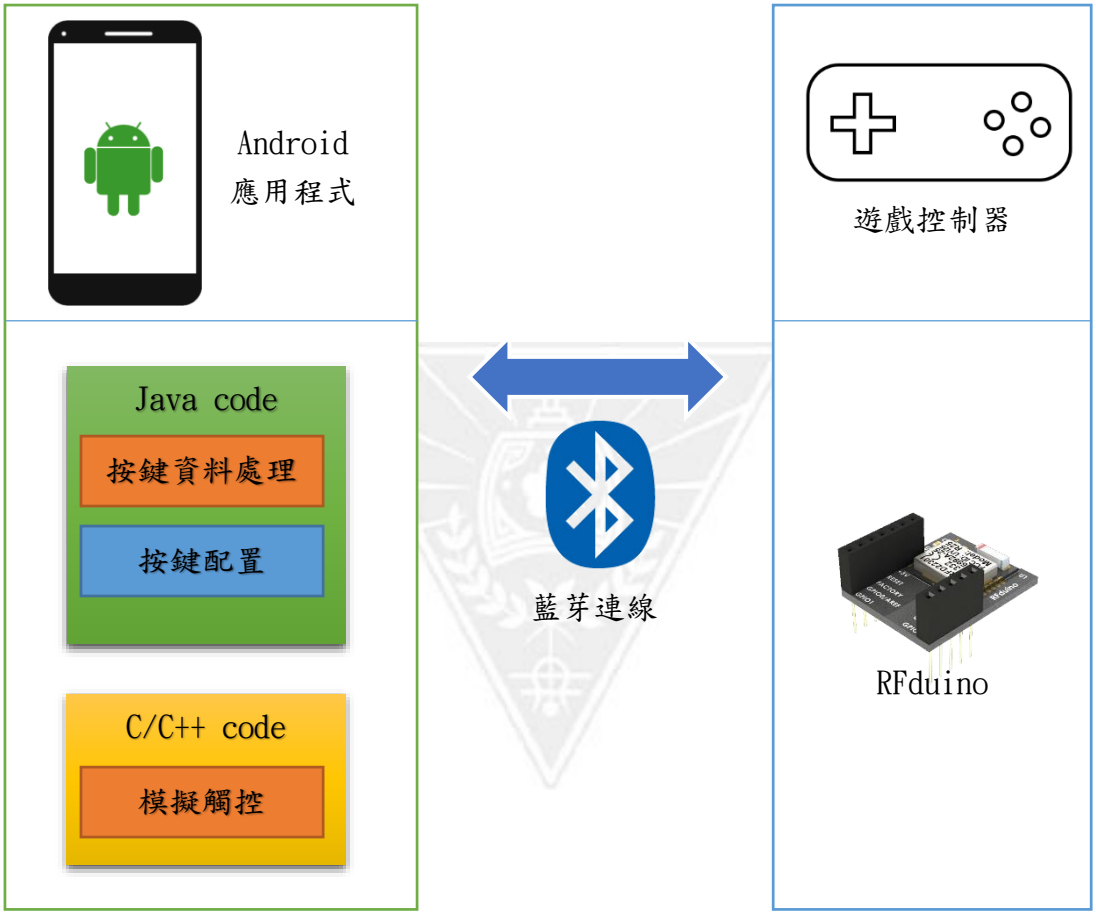


圖 3-1 系統架構圖

1. Android 應用程式：

Android 應用程式分為 Java 程式與 C/C++程式的部分，Java 程式有按鍵資料處理及按鍵配置，C/C++有模擬觸控。

i. 按鍵資料處理：

當遊戲控制器的按鍵按下時，本研究 Android 應用程式會接收遊戲控制器以藍芽傳遞的按鍵資料，而判斷按鍵的狀態。

ii. 遊戲控制器配置：

應用程式為設定遊戲控制器上的按鍵對應於 Android 裝置螢幕上的位置。因各種遊戲的玩法以及觸控螢幕的區域不同，使用者在使用此應用程式時，需先手動擷取一張遊戲畫面之圖片，而於本系統應用程式端選取該遊戲圖片，接著設定遊戲控制器欲對應於螢幕之位置，完成後即可進行遊戲。

iii. 模擬觸控：

本研究 Android 應用程式接收到按鍵的訊息後，將會於螢幕對應遊戲控制器按鍵的位置，以模擬使用者觸控螢幕的方式，於 Android 智慧型裝置上觸發訊息。因此按下本研究遊戲控制器按鍵，就如同手指觸控到 Android 智慧型裝置。

2. 遊戲控制器：

因 RFduino 輕巧、處理速度較 Arduino UNO 來得快且內建藍芽功能，故本研究遊戲控制器以 RFduino 為核心。而遊戲控制器上附有按鈕及雙軸搖桿(Dual Axis XY Joystick)，並以 RFduino 上的藍芽與 Android 智慧型裝置連接。遊戲控制器按下按鍵後，將會傳送資料至本研究 Android 應用程式，告知本研究 Android 應用程式使用者按下的按鈕。

第三節 系統流程

本系統的系統流程如圖 3-2，使用者能在非連線的狀態下新增、讀取或修改遊戲控制器的按鍵配置，而使用遊戲控制器時需要以藍芽連線。

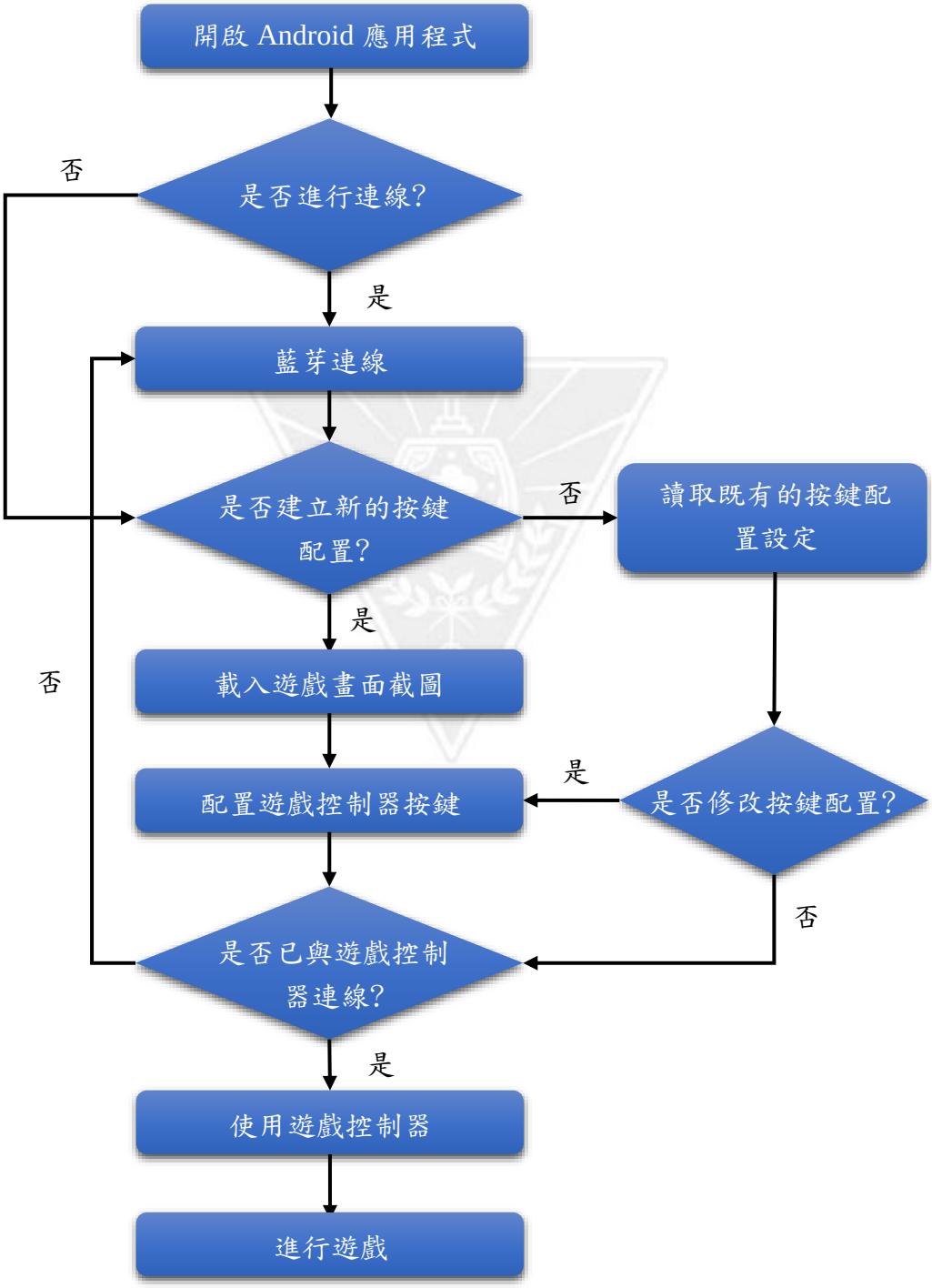


圖 3-2 系統流程圖

1. 藍芽連線：

   本系統以藍芽方式連接 Android 裝置與遊戲控制器。

2. 建立新的按鍵配置設定：

   使用者能建立新的按鍵配置設定來對應不同的遊戲。

3. 載入遊戲畫面截圖：

   各個遊戲的操控方式觸控位置皆不同，為使本系統能應用於各個遊戲，使用者須先擷取一張帶有遊戲觸控區的遊戲畫面，以便於配置遊戲控制器按鍵時使用。

4. 讀取既有的按鍵配置設定：

   當已建立按鍵配置設定時，使用者能讀取既有的按鍵配置設定。

5. 配置遊戲控制器按鍵：

   使用者能將遊戲控制器的按鍵，根據不同的遊戲操作的觸控區，對應於 Android 智慧型裝置螢幕上的位置。

6. 使用遊戲控制器：

   遊戲控制器會以藍芽向 Android 智慧型裝置傳遞按鍵的資料，Android 智慧型裝置接收到資料後，將會以使用者的按鍵配置設定於螢幕上模擬觸控，而達到手指於螢幕上滑動或是觸碰的效果。

7. 進行遊戲：

   完成配對與配置按鍵後，玩家可以開始進行遊戲，達到更好的遊戲體驗。

第四節 系統功能

基於上述的系統架構及系統流程，本研究所提供的系統功能有：

1. Android 模擬觸控：

   目前市面上的智慧型裝置遊戲控制器，皆是使遊戲來支援遊戲控制器，而本研究則是使遊戲控制器來支援遊戲，故本研究以模擬使用者觸控螢幕的方式對應遊戲控制器按鍵。以模擬觸控的方式，可使本研究遊戲控制器於 Android 平台上支援更多需要觸控螢幕的遊戲，甚至可以將本研究的遊戲控制器使用於非遊戲上。

2. Android 遊戲控制器配置應用程式：

   使用者可以選擇已儲存的遊戲控制器按鍵設定或是創建一個新的設定。載入已儲存的遊戲控制器按鍵設定以便快速使用，使用者也可以修改已儲存的遊戲控制器按鍵設定。若創建一個新的遊戲控制器按鍵設定，使用者載入遊戲畫面的螢幕截圖後，即可開始於畫面中對應遊戲控制器按鍵，配置完成後即可儲存並回到遊戲中遊玩。

3. 遊戲控制器於 Android 智慧型裝置中使用：

   使用者能於 Android 智慧型裝置中使用遊戲控制器，使得某些需要手指不斷於螢幕滑動的遊戲，以遊戲控制器來代替手指滑動。

4. 藍芽連線：

   遊戲控制器與 Android 智慧型裝置是以藍芽建立連線，而非以有線的方式連接。當使用者將 Android 智慧型裝置連接於電視或是外接螢幕時，可以將 Android 智慧型裝置擺放至更遠的距離。

第四章　系統實作與結果分析

本章共有兩節，第一節講述本研究系統實作的方法，而第二節說明並分析本研究的研究結果。

第一節 系統實作

本節分三部份，分別有模擬觸控、遊戲控制器配置應用程式以及遊戲控制器本身，分別對每個部份的設計做詳細的說明。

- Android 模擬觸控

  於 Android 的模擬觸控達到的方法有三種，分別為導入 Android 原始碼的 API、使用 Instrumentation object 及直接呼叫底層 Linux 的事件。

  1. 導入 Android 原始碼的 API：

     Android SDK 中的 API 並沒有提供模擬觸控的功能，所以需要導入 Android 原始碼中的 IWindowManager 類別，運用 IWindowManager 類別中的 injectKeyEvent / injectPointerEvent 來達到模擬觸控。

  2. 使用 Instrumentation：

     Instrumentation 是根據導入 Android 原始碼的 IWindowManager 類別所製作的。

  3. 直接呼叫底層 Linux 的事件：

     直接在 /dev/input/event(0-9) 做 Event Injection。

  導入 Android 原始碼的 API 與使用 Instrumentation object 這兩種方法想要在不同應用程式下也執行模擬觸控的功能時，會遇到 requires INJECT_EVENTS permission 的問題。要解決 requires INJECT_EVENTS permission 的問題需要特別去處理權限。而直接呼叫底層 Linux 的事件，只要將應用程式賦予超級使用者權限，即可於不同應用程式中實現模擬觸控的功

能。而本研究實現模擬觸控的功能所使用的方法為直接呼叫底層 Linux 的事件。

當使用者按下或觸碰到螢幕的元件時，會於 /dev/input/event(0-9) 中寫入一個相對應的訊號，而 Android 不斷的讀取 /dev/input/event(0-9) 中的事件，再將讀取到的訊息傳送到 WindowManagerServer，最後再從 WindowManagerServer 傳送資訊給對應的 ViewGroup 與 View。所以只要於 /dev/input/event(0-9) 中寫入訊號，就可以達到模擬觸控的功能。

Android SDK 中的 adb shell 提供了可以讀取 /dev/input/event(0-9) 中資訊的方法。只要連接 Android 裝置，並使用 adb shell 中的 getevent，就可以取得 /dev/input/event(0-9) 中的資訊。

如圖 4-1 所示，/dev/input/event0 的名稱為" h2w headset"，處理著耳機的事件；/dev/input/event2 處理鍵盤的事件；/dev/input/event3 是光感應器；/dev/input/event5 處理觸碰螢幕的事件。

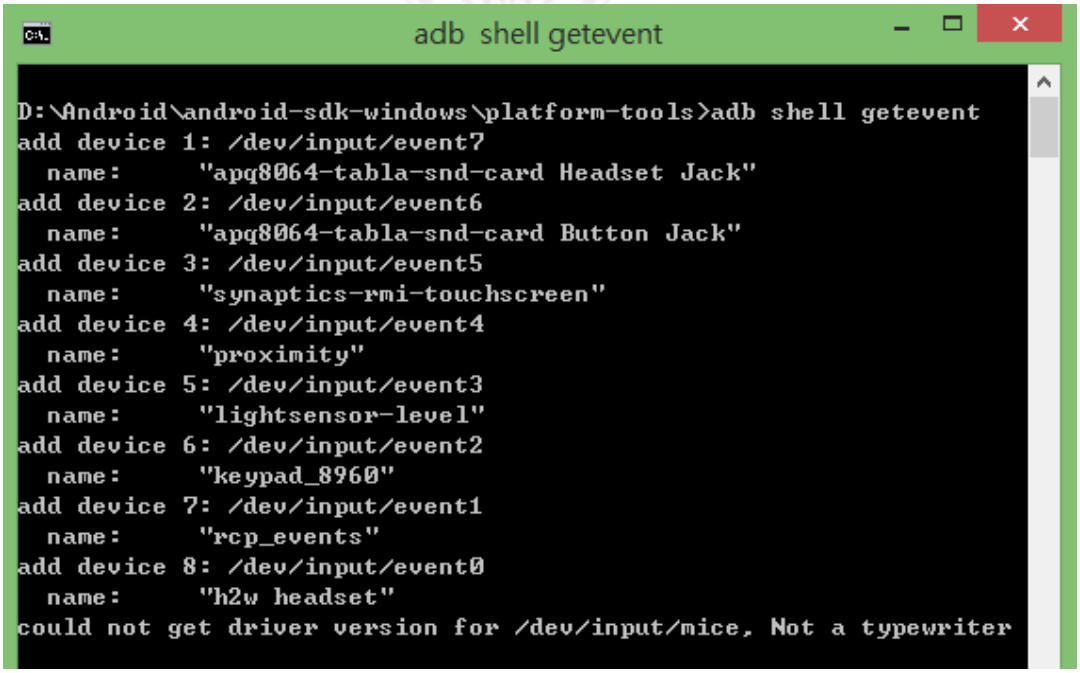


圖 4-1 Android 取得事件輸出

本研究需要實現模擬觸控的功能，所以需要向 /dev/input/event5 寫入資訊。Android SDK 中 adb shell 的 getevent 也可以針對某個事件來監聽。當手指觸碰螢幕的某個位置，所取得的資訊如圖 4-2 所示，每一行分為三段，分別為 type、code 與 value。

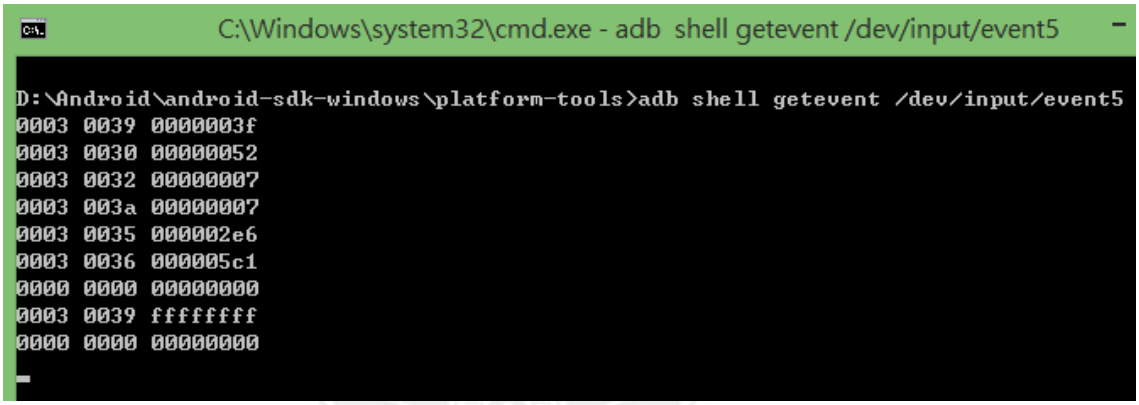


圖 4-2 Android 單點觸控事件輸出

type、code 與 value 定義於 linux\input.h 中，故得到的資訊可以從 linux\input.h 中找到數字所對應的定義。圖 4-2 為一個單點觸碰到放開所得到的資訊；第一行為此次觸碰的 ID，可以當作一次觸碰的開始；第二行為手指觸碰螢幕的手指橢圓長軸；第三行為手指觸碰螢幕的手指橢圓短軸；第四行為觸碰的壓力；第五行為觸碰到螢幕的 X 座標；第六行為觸碰到螢幕的 Y 座標；第七行為事件的同步。當手指離開螢幕時，第八行與第九行才出現。由此可知，完成一次的單點觸碰，所取得的資訊會有九行。

如圖 4-3 所示，單點觸控滑動時會不斷出現觸碰的範圍、觸碰的螢幕座標、觸碰的壓力與事件的同步。

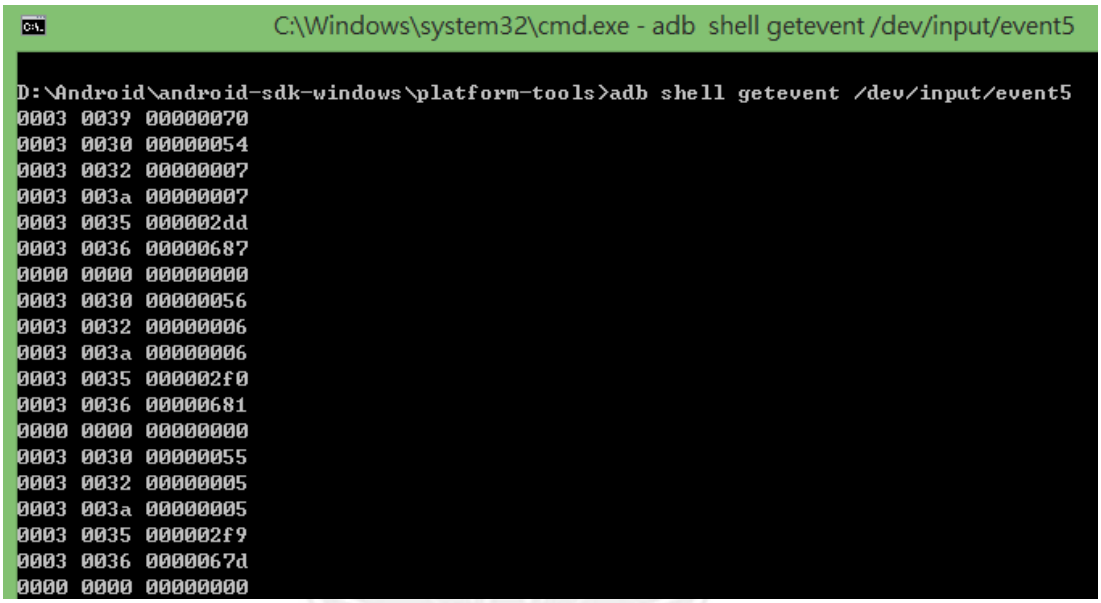


圖 4-3 Android 單點觸控滑動事件輸出

如圖 4-4 所示，多點觸控時，每多一點會多一個觸碰的 ID、觸碰的範圍、觸碰的螢幕座標、觸碰的壓力與事件的同步。

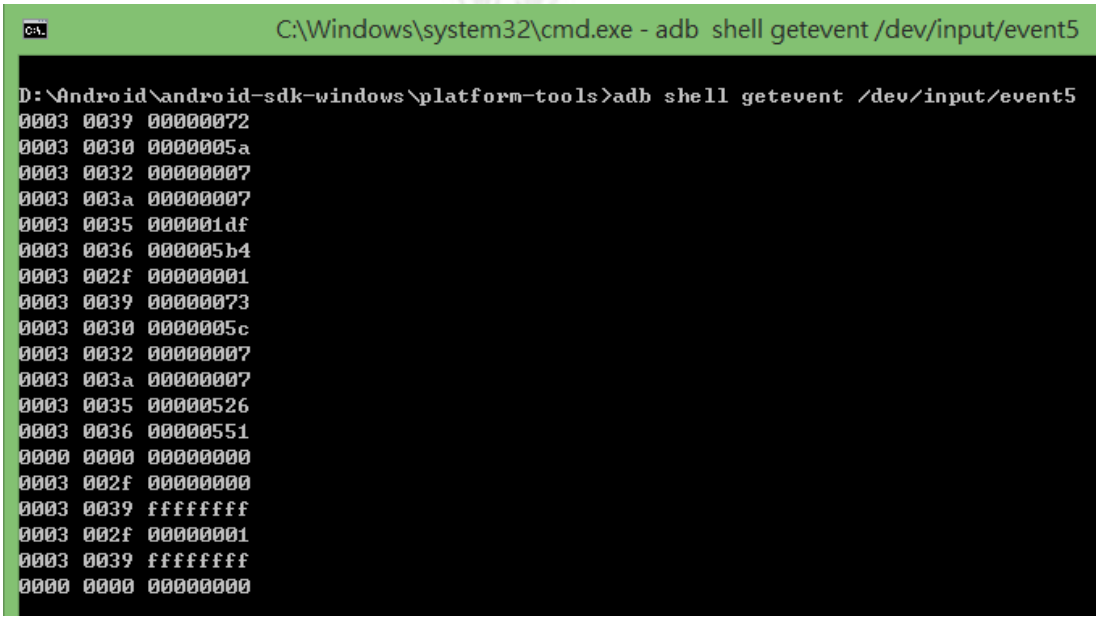


圖 4-4 Android 多點觸控事件輸出

使用 Android SDK 中 adb shell 的 getevent 時，得到的螢幕位置為十六進位的值，將十六進位轉為十進位所得到的數字並非如螢幕解析度下得到的數字，故要轉為螢幕解析度的數值，才能讓 Android 上的 GUI 使用。

如圖 4-5，本研究所使用的 Android 裝置解析度為 1080 * 1920，使用 adb shell 的 getevent –p 將會出現 X 座標的最小值為 0，最大值為 1600；Y 座標最小值為 0，最大值為 2695。

```
add device 3: /dev/input/event5
  name:     "synaptics-rmi-touchscreen"
  events:
    KEY (0001): 0066  008b  009e  00d9  00f9
    ABS (0003): 002a  : value 0, min 0, max -2042623353, fuzz 0, flat 0, resolution 0
                002b  : value 0, min 0, max 16711695, fuzz 0, flat 0, resolution 0
                002f  : value 0, min 0, max 9, fuzz 0, flat 0, resolution 0
                0030  : value 0, min 0, max 255, fuzz 0, flat 0, resolution 0
                0032  : value 0, min 0, max 30, fuzz 0, flat 0, resolution 0
                0035  : value 0, min 0, max 1600, fuzz 0, flat 0, resolution 0
                0036  : value 0, min 0, max 2695, fuzz 0, flat 0, resolution 0
                0039  : value 0, min 0, max 65535, fuzz 0, flat 0, resolution 0
                003a  : value 0, min 0, max 30, fuzz 0, flat 0, resolution 0
  input props:
    INPUT_PROP_DIRECT
```

圖 4-5 Android 觸控座標範圍

故 adb shell 的 getevent 得到的 X 座標 x、Y 座標 y、當前螢幕寬 Width 與當前螢幕高 Height，轉換為螢幕解析度的座標公式為：

$$x_{new} = \frac{(x - x_{min}) \times Width}{x_{max} - x_{min}}, \qquad y_{new} = \frac{(y - y_{min}) \times Height}{y_{max} - y_{min}}$$

當裝置轉為橫向時，螢幕解析度的 X 座標與 Y 座標將會互換，且 Android 的 Y 座標為 Y 向下。而 adb shell 的 getevent 得到的 X 座標與 Y 座標卻不會改變，因此座標轉換的公式將改變為：

$$x_{new} = \frac{(y - x_{min}) \times Width}{x_{max} - x_{min}}, \qquad y_{new} = Height - \frac{(x - y_{min}) \times Height}{y_{max} - y_{min}}$$

本研究模擬觸控的部分使用 Android NDK 所編寫的 C/C++ 程式製作了事件搜尋、記憶體控管、取得事件的詳細說明以及寫入事件。於 Android SDK 的 Java 程式可以像 Android SDK 中的 adb shell 一樣的輸入指令。

模擬觸控的流程如圖 4-6 所示，使用者於遊戲控制器配置應用程式配置好在螢幕上所對應的按鍵時，相對應的螢幕座標會被記錄下來，而使用者按下遊戲控制器的按鈕時，Java 程式端將座標及需要的資料傳送至 C/C++ 程式中執行觸控。

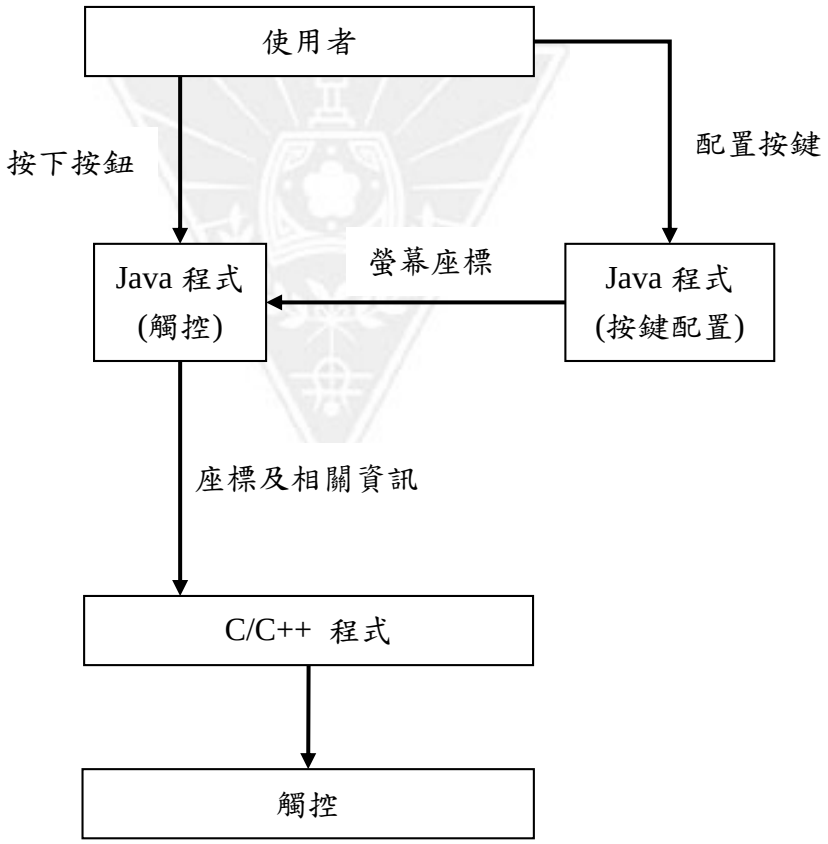


圖 4-6 模擬觸控流程

- Android 遊戲控制器配置應用程式

本研究 Android 遊戲控制器配置應用程式如圖 4-7 所示，分為藍芽連接與配置遊戲控制器按鍵。

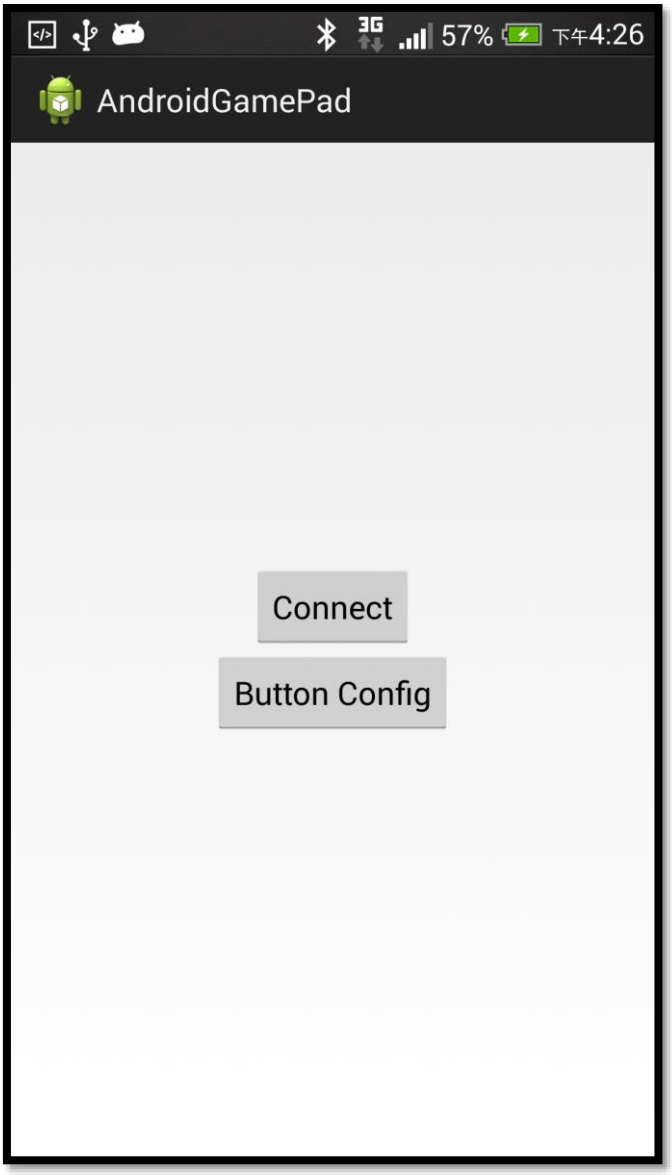


圖 4-7 Android 遊戲控制器配置應用程式圖

藍芽連接的功能有搜尋並連接到遊戲控制器，以及斷開連接的功能。遊戲控制器按鍵配置的功能有載入圖片、配置按鍵、儲存設定與修改已儲存的設定。

遊戲控制器按鍵配置流程如圖 4-8 所示，使用者可以選擇已儲存的遊戲控制器按鍵設定或是創建一個新的設定。載入已儲存的遊戲控制器按鍵設定以便快速使用，使用者也可以修改已儲存的遊戲控制器按鍵設定。若創建一個新的遊戲控制器按鍵設定，使用者載入遊戲畫面的螢幕截圖後，即可開始於畫面中對應遊戲控制器按鍵，配置完成後即可儲存並回到遊戲中遊玩。

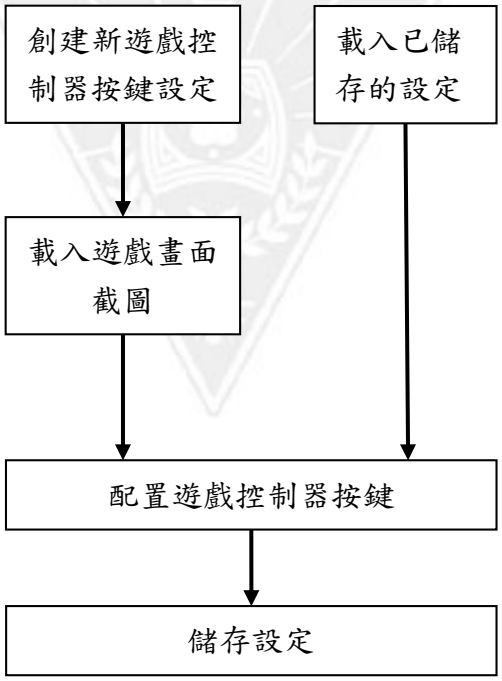


圖 4-8 遊戲控制器按鍵配置流程圖

- 遊戲控制器

由於 RFduino 本身可用於讀取或輸出的腳位只有七個，而本研究遊戲控制器的按鈕不只六個，所以按鈕的不分需要另行設計。如圖 4-9 所示，開關與電阻的並聯串聯在一起。

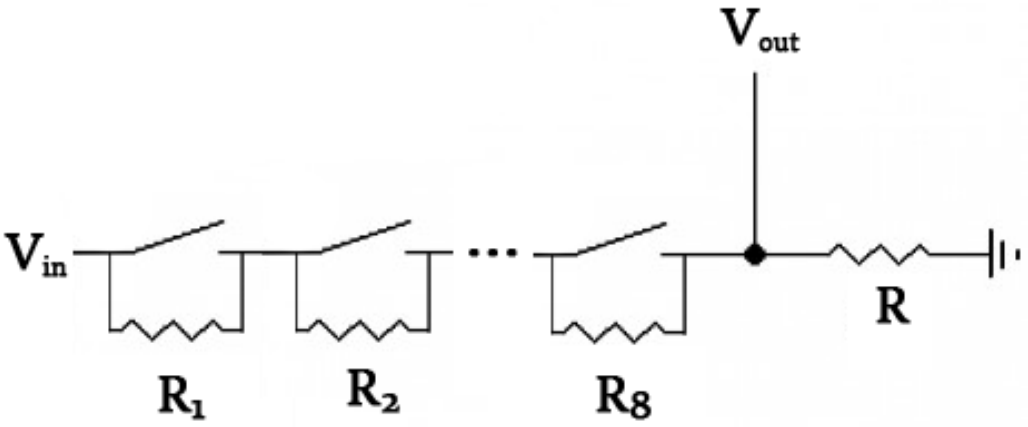


圖 4-9 按鈕連接方式示意圖

當電流從 V_{in} 進入時，若開關全放掉會經過所有的電阻，而按下一個開關就會跳過一個電阻。而 V_{out} 連接到 RFduino 上，當開關全放掉時，因電經過所有的電阻，所以讀取到的值會接近 0，而所有的開關都按下時，讀取到的值會接近 1023。本研究藉著電阻的改變來判斷開關是否按下，而非以往以讀取 0 與 1 的方式判斷開關的狀態。

本研究使用的雙軸搖桿(Dual Axis XY Joystick)如圖 4-10 所示，有一個接地、一個接電源、一個 X 軸的訊號、一個 Y 軸的訊號以及按鈕的訊號。

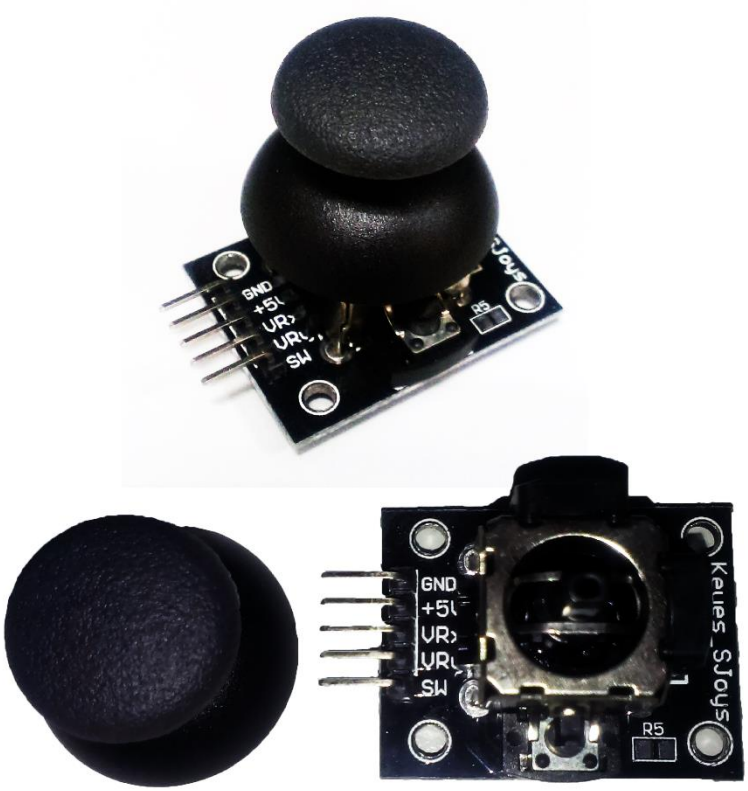


圖 4-10 雙軸搖桿圖

使用兩個雙軸搖桿需要使用到七個腳位來讀取訊號，而 RFduino 本身可用於讀取或輸出的腳位只有七個。因此將雙軸搖桿的按鈕就與其他按鈕串在一起，而達到節省腳位的目的。

- 藍芽連線

本研究藍芽連接的功能有搜尋並連接到遊戲控制器，以及斷開連接的功能。而藍芽連線分為 Android 應用程式與遊戲控制器。

1. Android 應用程式：

如圖 4-11，當使用者開啟本研究 Android 應用程式時，程式會驗證使用者的裝置是否支援藍芽設備。若支援藍芽設備，則本研究 Android 應用程式將開啟藍芽服務，並尋找藍芽設備。搜尋到藍芽設備後，本研究 Android 應用程式會彈出可連接的設備視窗，等待使用者點擊後連接。與藍芽設備連接後，將開始不斷的接收訊息。

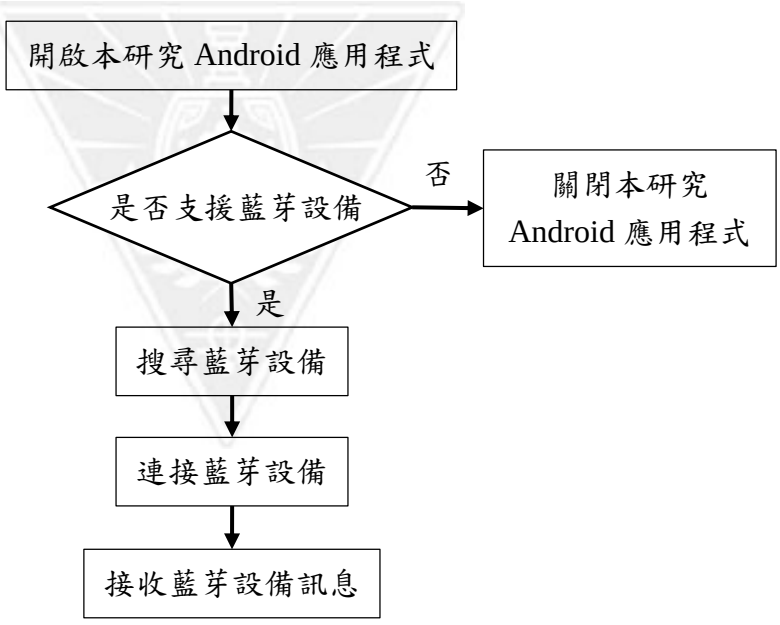


圖 4-11 本研究 Android 應用程式藍芽連線流程圖

2. 遊戲控制器：

本研究遊戲控制器部分將不斷的傳送九位數的整數。由左至右的前三位數為按鈕按下時改變電阻所產生的數值，而中間三位數為雙軸搖桿左右移動所改變的數值，右邊三位數為雙軸搖桿上下移動所改變的數值。

- 系統測試

圖 4-12 所示，本研究以四顆按鈕、一顆 LED 燈及一個雙軸搖桿作為遊戲控制器的測試版本。系統測試前，於 Android 智慧型裝置中的設定下的開發人員選項，將「顯示觸控回應」的選項勾起。當遊戲控制器按下按鍵或是移動雙軸搖桿時，Android 智慧型裝置的螢幕上將會顯示觸控的回應。

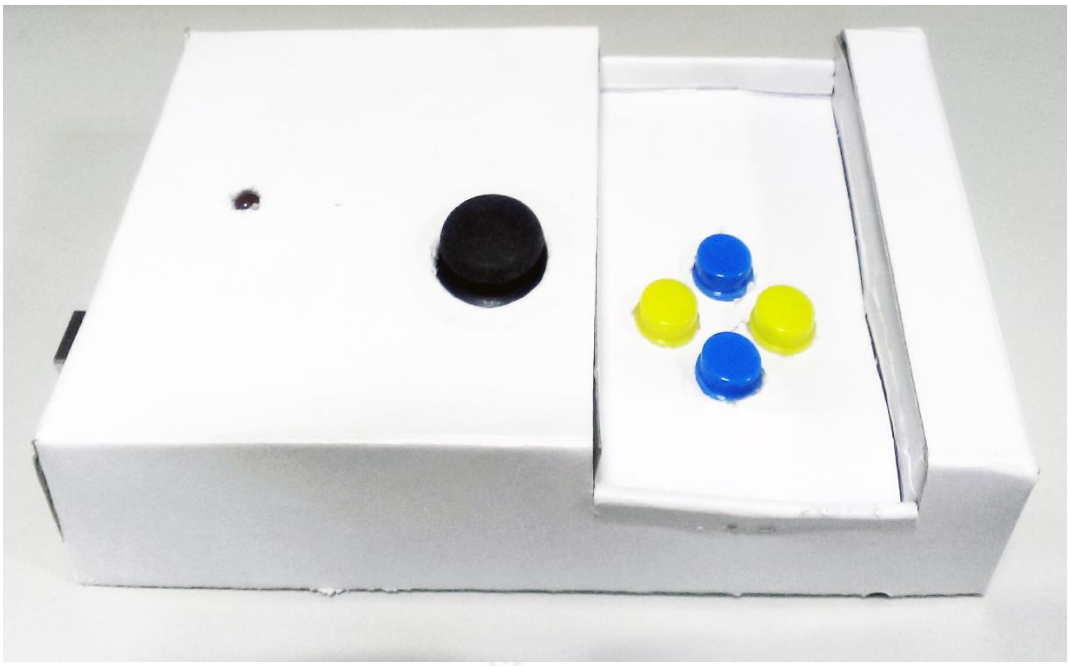


圖 4-12 遊戲控制器測試版本

本研究系統測試的步驟有 7 項，分別測試遊戲控制器與 Android 遊戲控制器應用程式之間互動的情況。

1. 連接 Android 智慧型裝置與遊戲控制器。
2. 於遊戲控制器上按下個個按鈕以及使用雙軸搖桿。
3. 擱置遊戲控制器一段時間。

4. 開啟任一款需要觸控螢幕的遊戲，於 Android 遊戲控制器應用程式中設定按鍵所要對應於 Android 智慧型裝置螢幕上的觸控區域。
5. 使用遊戲控制器來體驗遊戲。
6. 將遊戲控制器與 Android 裝置的距離拉長。
7. 斷開 Android 智慧型裝置與遊戲控制器的連接。

本研究測試的項目如表 4-1 所示，當 Android 應用程式連接至遊戲控制器時，遊戲控制器上的 LED 燈將會亮起，反之取消連線時，LED 燈即不亮。按下遊戲控制器的按鈕後，於 Android 智慧型裝置上立即有觸控螢幕的反應，感覺不出有延遲的情形。移動雙軸搖桿時，Android 觸控位置於自訂的圓圈範圍內能自由的移動。將遊戲控制器擱置一段時間，Android 裝置上沒有出現任何的觸控反應。將遊戲控制器與 Android 裝置的距離拉長後約 25 公尺，才出現無法連接的狀況。

表 4-1 系統測試項目表

測試項目	測試結果
LED 燈狀態	Android 應用程式連接至遊戲控制器時，遊戲控制器上的 LED 燈將會亮起，斷開連線時，LED 燈不亮。
延遲情況	按鈕按下後，Android 智慧型裝置上立即有觸控螢幕的反應。
雙軸搖桿移動狀況	於自訂的圓圈範圍內，能自由移動。
是否有雜訊	擱置遊戲控制器時，Android 智慧型裝置上無任何觸控螢幕的反應。
最遠距離	約 25 m

第二節 結果分析

本節將市面上既有並與本研究相似的智慧型裝置遊戲控制器，與本研究的 Android 智慧型裝置遊戲控制器分析比較。

表 4-2 Android 智慧型裝置遊戲控制器比較表

	本研究遊戲控制器	Samsung Android Wireless Gamepad	Gametel	Logitech PowerShell
模擬觸控	有	無	無	無
遊戲開發商配合遊戲控制器	不需要	需要	需要	需要
藍芽版本	4.0	3.0	3.0	無
支援平台	Android 4.3 以上	Android 4.1 以上	Android 2.1 以上、iOS 4 以上、Windows 7 以上、Mac OS X Snow Leopard 10.6 以上	iOS 7
價格	$22.99	£59.00	$19.95	NTD 1990

如表 4-2 所示，本遊戲控制器藉模擬觸控來支援遊戲，能夠支援更多需要觸控的遊戲甚至不只是遊戲。其他遊戲控制器開發商需要與遊戲開發者合作，才能使遊戲支援遊戲控制器。藍芽連接的部分，本研究使用的版本為功耗更低的藍芽 4.0，其餘的則是較古老的藍芽 3.0 版本。

本研究遊戲控制器所花費的金額為 22.99 美金的 RFduino 加上不到台幣一百元的按鍵與雙軸搖桿，就能支援相當大量的遊戲與應用程式。雖然 Gametel 的價格較本研究遊戲控制器低，但 Gametel 所支援的 Android 遊戲只有約 80 款，而本研究的遊戲控制器若量產，可能更接近 Gametel 的價格。Samsung Android Wireless Gamepad 的價格為 59 歐元，目前只支援約 35 款。Logitech PowerShell 為台幣 1990 元，遊戲控制器開發商只要配合 iOS 7 的規範，就能在支援 iOS 遊戲控制器的 iOS 遊戲中使用。

第五章　結論與未來建議

第一節 結論

許多遊戲於行動裝置上遊玩時，操作起來並非那麼的方便。雖然市面上已有支援智慧型裝置的遊戲控制器，但既有的遊戲控制器只支援部分的遊戲。且遊戲控制器若要在遊戲上進行，需要遊戲開發者另外配合遊戲控制器開發商。本研究藉模擬觸控來支援遊戲，因此不需要遊戲開發商的配合，而能支援更多需要觸控的遊戲。

本研究的遊戲控制器，使用模擬使用者手指觸控智慧型裝置螢幕的方式，使得遊戲控制器將能支援更多需要觸控的遊戲，甚至不限定只能於遊戲中使用。使用遊戲控制器代替手指於螢幕的觸控，以解決會流手汗的玩家，不停的滑動造成的觸控的感應靈敏度下降以及手指的不適。遊戲控制器與 Android 智慧型裝置是以藍芽建立連線，而非以有線的方式連接。將 Android 智慧型裝置連接於電視或是外接螢幕時，可使 Android 智慧型裝置擺放至更遠的距離。

簡而言之，於 Android 智慧型裝置使用模擬使用者觸控的遊戲控制器，不但能支援更多的遊戲，還能降低手指過度頻繁於智慧型裝置螢幕滑動，而解決會流手汗的玩家，不停的滑動造成的觸控的感應靈敏度下降與手指的不適。而本研究遊戲控制器的出現，將使遊戲設計者不再受限於遊戲控制器開發商，開發遊戲時也將更為簡單。

第二節 未來建議

本節在未來發展改進之方向提出以下五點：

- 將模擬觸控的功能設為選項：

  本研究遊戲控制器預設為模擬觸控，若能與遊戲開發商合作，則遊戲控制器將支援更多的 Android 裝置，使得無法修改應用程式權限的 Android 裝置也能使用本遊戲控制器。

- 跨平台支援：

  本身支援開發遊戲控制器的 iOS 7 於 iOS 裝置中日益普及，若遊戲控制器能跨平台使用，則能支援更多的遊戲，以及使更多的使用者來使用。

- 能擴充按鍵：

  本研究的遊戲控制器目前只配置了四顆按鈕與一個雙軸搖桿，若 Android 遊戲操作超過四個觸控區域，則本研究遊戲控制器將無法支援。建議在往後開發的遊戲控制器能讓使用者擴充按鍵數量，以操作需要更多點觸控的遊戲。

- 更多點觸控：

  目前本研究遊戲控制器只支援至兩點觸控，已能操作大部分需要觸控的遊戲。未來可以提供更多點的觸控，以操作需要更多點同時觸控的遊戲。

- 更好的遊戲控制器外觀：

  本研究著重於遊戲控制器的功能上，而外觀上並無特別設計。因 RFduino 輕巧體積小，未來可以於外觀上設計的更為人性化。

參考文獻

一、 中文部分

余瀧濱. (2012 年 6 月). 於 Android 智慧型手機以近場通訊啟用藍芽資料傳輸之研究.

林城. (2011). Google Android 應用程式開發實戰 第三版 適用 Android SDK 2.x/3.x. 碁峰資訊.

金純, 許光辰, & 孫睿. (2002). 藍芽技術. 五南書局.

禹帆. (2001). 無線藍芽技術的深入探討. 文魁資訊.

陳筠錚. (2005 年 6 月). 應用藍芽協定之行動導覽服務.

二、 英文部分

Arduino. (n.d.). Retrieved from http://arduino.cc/

Arduino Language Reference. (n.d.). Retrieved from http://arduino.cc/en/Reference/HomePage

Arduino Uno. (n.d.). Retrieved from http://arduino.cc/en/Main/ArduinoBoardUno

Fructel. (2012). *Specification.* Retrieved from Gametel: http://gametel.se/index.php/explore/specification

Game Controller Programming Guide. (2013). Retrieved from iOS Developer Library: https://developer.apple.com/library/ios/documentation/ServicesDiscovery/Conceptual/GameControllerPG/GameControllerPG.pdf

Liang, S. (1999). *The Java™ Native Interface : Programmer's Guide and Specification.*

Paul, I., & PCWorld. (2010, 10 26). *Wi-Fi Direct vs. Bluetooth 4.0: A Battle for Supremacy*. Retrieved from PCWorld: http://www.pcworld.com/article/208778/Wi_Fi_Direct_vs_Bluetooth_4_0_A_Battle_for_Supremacy.html

POWERSHELL CONTROLLER + BATTERY Setup Guide. (2013). Retrieved from Logitech Support + Downloads: http://www.logitech.com/assets/50209/7/powershell-quick-start-guide.pdf

Quick Start Guide (Preliminary DRAFT). (2013, 10 10). Retrieved from RFduino: http://files.rfdigital.com/rfduino.quick.start.guide.pdf

RFduino. (2013). Retrieved from RFduino: https://ssl.rfduino.com/

RFduino DATASHEET. (2013, 12 5). Retrieved from RFduino: http://files.rfdigital.com/rfduino.datasheet.pdf

Samsung. (2014). *Android Wireless Gamepad*. Retrieved from http://www.samsung.com/uk/consumer/mobile-devices/smartphones/smartphone-accessories/EI-GP20HNBEGWW

Samsung. (2014). *S Console*. Retrieved from SAMSUNG Developers: http://developer.samsung.com/s-console

Shrunk down an Arduino to the size of a finger-tip! (2013, 4 5). Retrieved from Kickstarter: http://www.kickstarter.com/projects/1608192864/rfduino-iphone-bluetooth-40-arduino-compatible-boa?ref=live